

Lesson: Evaluating Model Performance

Introduction

In our previous lessons, we explored how to build models for classification (e.g., Logistic Regression, k-NN, Decision Trees) and for prediction using regression techniques (Linear, Polynomial, Ridge, and Lasso). But building a model is only half the journey. The other half, and questionably the more important one, is answering this critical question:

How well is my model performing?

Whether we are predicting house prices or diagnosing a disease, knowing if the model's predictions can be trusted is essential. A high-performing model isn't just one that fits the training data well, it generalizes well to **unseen** data.

That's why this lesson is dedicated to **evaluating model performance**, using the right metrics and validation techniques.

Learning Outcomes

By the end of this lesson, you will be able to:

- Understand and apply key **classification metrics**: Accuracy, Precision, Recall, F1-Score, ROC-AUC.
 - Evaluate **regression models** using: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score.
 - Understand the concept and importance of **cross-validation** for building robust models.
-

What Are Evaluation Metrics?

Evaluation metrics are numerical indicators utilized to evaluate the performance and effectiveness of statistical or machine learning models. These metrics offer insights into the model's performance and assist in comparing various models or algorithms.

Classification Metrics

When building classification models, making predictions is just the beginning. To truly understand how well our model performs, we need to go beyond accuracy and evaluate how it performs across different classes, especially when the data is imbalanced. Classification metrics help us answer key questions like:

- How often is the model correct?
- Is it better at identifying positives or negatives?
- Does it miss important cases?

We'll break down essential metrics like **confusion matrix**, **accuracy**, **precision**, **recall**, **F1-score**, and **ROC-AUC** to give you a well-rounded view of your model's strengths and weaknesses.

Let's define some key terms first

1. Confusion Matrix

A confusion matrix is an N x N matrix used to evaluate the performance of a classification algorithm, where N represents the number of predicted classes. It effectively visualizes and summarizes this performance.

Here are a few definitions you need to remember for a confusion matrix:

When evaluating classification models, predictions fall into one of four categories based on the actual and predicted values:

- **True positive:** The model predicted positive, and the actual value is also positive.
- **True negative:** The model predicted negative, and the actual value is also negative.
- **False positive (Type 1 error):** The model predicted positive, but the actual value is negative.
- **False negative (Type 2 error):** The model predicted negative, but the actual value is positive.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Example: Spam Classifier

Imagine you're building a simple spam classifier. After testing it on a set of emails, you get the following outcomes:

- You correctly classified 50 spam emails as spam (True Positives).
- You correctly identified 40 legitimate emails as not spam (True Negatives).
- You wrongly flagged 10 legitimate emails as spam (False Positives).
- You failed to detect 5 spam emails and marked them as not spam (False Negatives).

This results in the following confusion matrix:

	Predicted: Positive	Predicted: Negative
Actual Positive	50	5
Actual Negative	10	40

Code

```

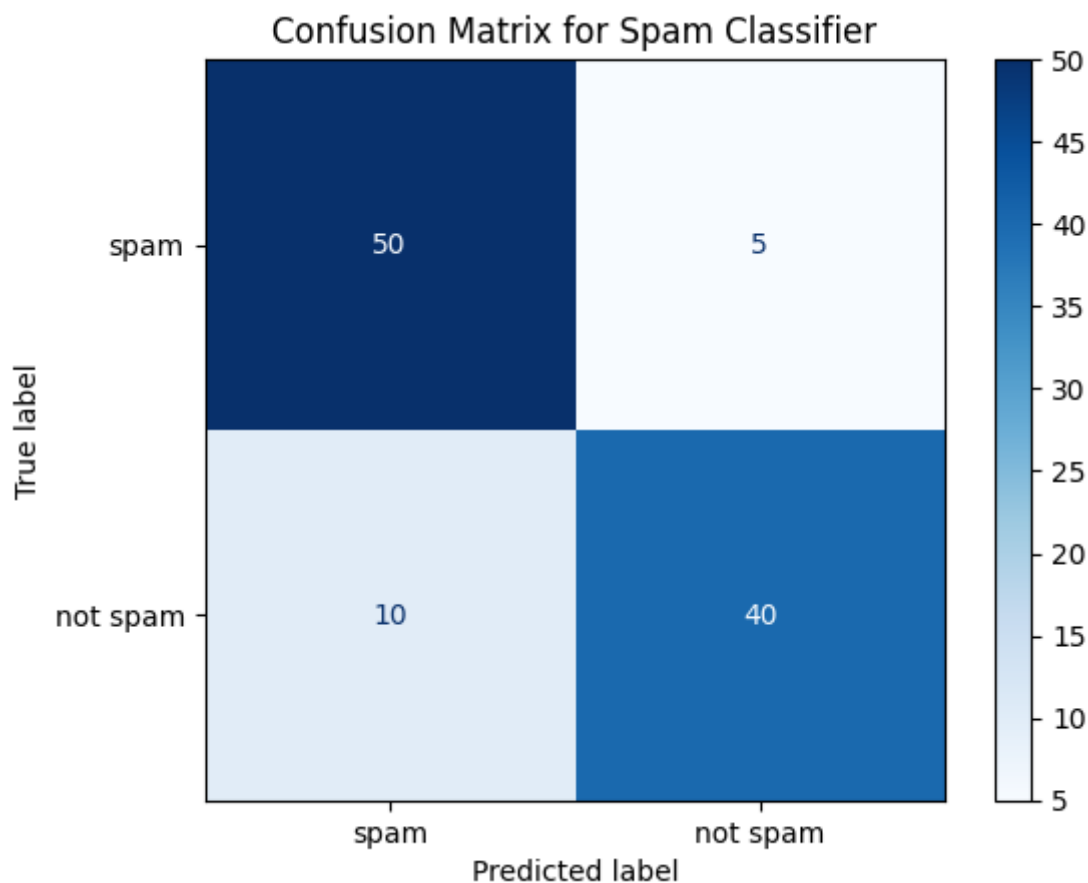
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Simulated ground truth and predictions
y_true = ['spam'] * 55 + ['not spam'] * 50
y_pred = ['spam'] * 50 + ['not spam'] * 5 + ['spam'] * 10 + ['not spam'] * 40

# Compute the confusion matrix
cm = confusion_matrix(y_true, y_pred, labels=['spam', 'not spam'])

# Display it visually
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['spam', 'not spam'])
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix for Spam Classifier")
plt.show()

```



Accuracy

Accuracy is a common evaluation metric for classification tasks. It measures the proportion of correct predictions made by the model out of all predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

For example, a classifier made the following predictions:

	Predicted: Positive	Predicted: Negative
Actual: Positive	True Positive (TP) = 50	False Negative (FN) = 5
Actual: Negative	False Positive (FP) = 10	True Negative (TN) = 35

Accuracy = $\frac{50+5}{50+35+10+5} = \frac{85}{100} = 0.85$. So, the model is 85% accurate

Interpretation

An accuracy of approximately 85.7 percent means that out of all predictions made, the model correctly classified about 86 percent of the data points.

While accuracy gives a quick overall snapshot of model performance, it may not be reliable when the dataset is imbalanced (for example, when one class is much more common than the other). In such cases, metrics like precision, recall, or F1 score may offer better insights.

Code example

```
from sklearn.metrics import accuracy_score, confusion_matrix

# Actual and predicted Labels
y_true = [1, 0, 1, 1, 0, 1, 0, 0, 1, 0]
y_pred = [1, 0, 1, 0, 0, 1, 0, 1, 1, 0]

# Accuracy
accuracy = accuracy_score(y_true, y_pred)
print(f"Accuracy: {accuracy:.2f}")
```

Precision

Precision measures how many of the instances predicted as positive by the model are actually positive. It is useful when the cost of false positives is high.

$$\text{Precision} = \frac{TP}{TP + FN}$$

Recall

Recall measures the model's ability to correctly identify all actual positive cases. It is especially important when missing a positive case has a high cost.

$$\text{Recall} = \frac{TP}{TP + FP}$$

Sure. Here's a clean explanation of **precision** and **recall**, an example using a confusion matrix, and a brief interpretation of their physical meaning:

Precision and Recall

- Precision: How many of the predicted positives were actually positive?
- Recall: How many of the actual positives did we correctly identify?

Example

Suppose we have the following classification results:

- True Positives (TP) = 50
- False Positives (FP) = 10
- False Negatives (FN) = 5

We can now compute:

- Precision = $50 / (50 + 10) = 0.8333$
- Recall = $50 / (50 + 5) = 0.9091$

Interpretation

- A **precision of 0.8333** means that out of all the times the model predicted "positive", about 83 percent were correct. High precision means fewer false alarms.
- A **recall of 0.9091** means that the model was able to detect about 91 percent of all actual positive cases. High recall means the model misses fewer true positives.

In practical terms, precision is important when false positives are costly (for example, in spam detection), while recall is critical when missing a true positive has serious consequences (like detecting a disease).

Code example:

```
from sklearn.metrics import precision_score, recall_score

# Actual and predicted Labels
y_true = [1]*50 + [1]*5 + [0]*10 + [0]*35
y_pred = [1]*50 + [0]*5 + [1]*10 + [0]*35

# Precision and recall
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)

print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
```

Use Precision when false positives are costly, Recall when false negatives are costly.

F1-Score

The F1 score is a measure that combines both precision and recall. Unlike precision and recall, which focus on only one type of error, the F1 score takes into account both false positives and false negatives. This makes the F1 score more informative than accuracy, especially when dealing with uneven class distributions.

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall}$$

The F1 score is a better choice than accuracy when the costs of false positives and false negatives are both important and when the data has an uneven class distribution.

Example:

- $Recall = \frac{50}{50+5} = 0.9091$
- $F1 = \frac{2*(0.8333*0.9091)}{0.8333+0.9091} \approx 0.8696$

Interpretation

The F1 score of approximately 0.8696 means the model has a good balance between precision and recall. It performs well in identifying true positives while keeping both false positives and false negatives relatively low.

This score is especially helpful when the dataset is imbalanced, and you want to ensure the model is not only accurate overall but also reliable in detecting the minority class. The F1 score becomes a single, more meaningful metric when neither precision nor recall alone gives the full picture.

Code example:

```
from sklearn.metrics import f1_score

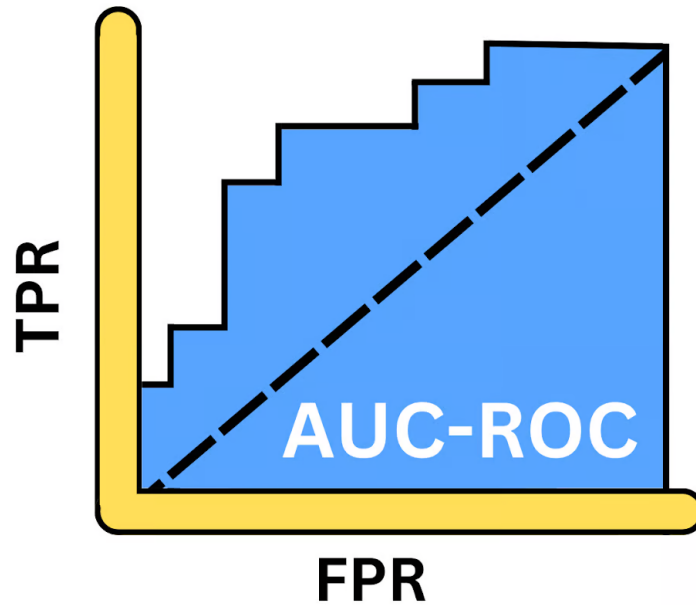
# Calculate F1 score
f1 = f1_score(y_true, y_pred)

print("F1-Score:", round(f1, 4))
# F1-Score: 0.8696
```

ROC-AUC (Receiver Operating Characteristic - Area Under Curve)

The ROC curve visually represents the trade-offs between the true positive rate (TPR) and the false positive rate (FPR) at various thresholds. It provides insights into how well a model can balance the detection of positive instances while minimizing false positives across these thresholds. The AUC, or Area Under the Curve, is a single scalar value ranging from 0 to 1, which provides a performance snapshot of the model. The AUC is calculated after generating the ROC curve, as it represents the area beneath the curve.

This metric is especially useful for comparing models and assessing performance when class distributions are imbalanced.



True Positive Rate (TPR)

Also known as **Recall** or **Sensitivity**, the true positive rate measures the proportion of actual positives that were correctly identified by the model.

$$TPR = \frac{TP}{TP + FN}$$

It tells us how well the model catches positive cases.

False Positive Rate (FPR)

The false positive rate measures the proportion of actual negatives that were incorrectly classified as positive.

$$FPR = \frac{FP}{FP + TN}$$

It reflects how often the model incorrectly raises a false alarm.

By plotting TPR against FPR at different thresholds, the ROC curve illustrates the trade-off between sensitivity and the likelihood of false alarms. This helps in choosing the best threshold for the model based on the problem's tolerance for errors.

Advantages of ROC-AUC:

Measuring Performance Across Thresholds

- AUC-ROC evaluates model performance across all classification thresholds.
- Unlike metrics like accuracy, precision, or F1-score, it is threshold-independent.

Model Comparison

- Provides a single scalar value for easy comparison of multiple models.
- Helps fairly compare models with different optimal thresholds.

Handling Class Imbalance

- More reliable in imbalanced datasets (e.g., fraud detection, medical diagnosis).
- Accuracy can be misleading in such cases, while AUC-ROC offers a balanced assessment.

Code example:

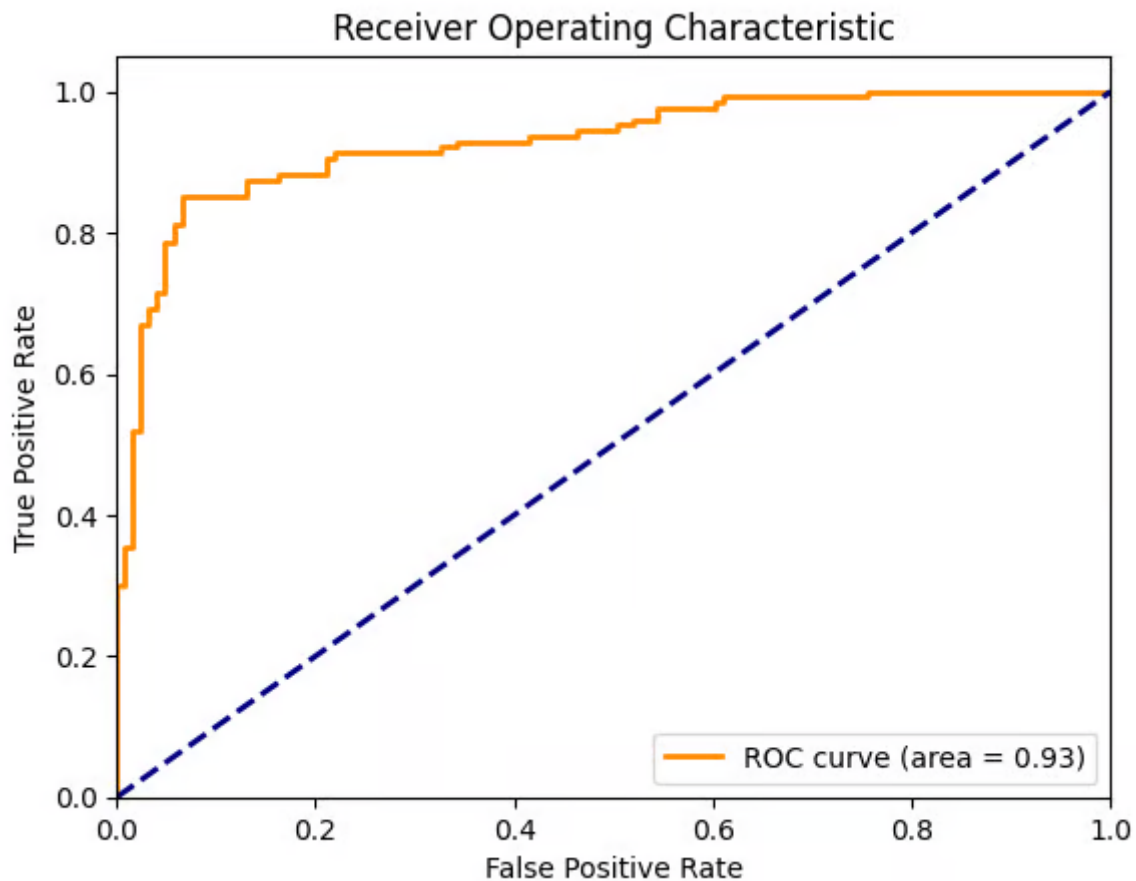
```
from sklearn.metrics import roc_auc_score
# Compute ROC AUC score
roc_auc = roc_auc_score(y_true, y_scores)
print("ROC AUC Score:", round(roc_auc, 4))
```

Plotting the ROC Curve

```
```python
fpr, tpr, thresholds = roc_curve(y_test, y_probs)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc="lower right")
plt.show()
```





Metric	Best Use Case	Strengths	Limitations
Accuracy	Balanced class distribution	Simple, intuitive	Misleading with imbalanced classes
Precision	False positives are costly (e.g., spam filter)	Focuses on correct positive predictions	Ignores false negatives
Recall	False negatives are costly (e.g., disease detection)	Captures all actual positives	Ignores false positives
F1-Score	You need balance between precision and recall	Balances both metrics	Harder to interpret directly
ROC-AUC	Comparing classifiers across thresholds	Threshold-independent, works well with imbalance	Requires probability scores

## Regression Metrics

In regression tasks, we're not predicting categories, we're predicting **continuous values** like house prices, temperatures, or stock prices. So, how do we know if our model is doing a good job?

Regression metrics help us quantify **how far off our predictions are** from the actual values. We'll explore key metrics such as:

- **Mean Squared Error (MSE)** : penalizes larger errors more heavily.
- **Root Mean Squared Error (RMSE)**: provides error in original units.

- **R<sup>2</sup> Score** : tells us how well our model explains the variability in the data.

## Mean Squared Error (MSE)

Mean squared error is perhaps the most popular metric used for regression problems. It essentially finds the average of the squared difference between the target value and the value predicted by the regression model.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where:

- $y_j$ : ground-truth value
- $\hat{y}_i$ : predicted value from the regression model
- $N$ : number of datums

## Root Mean Squared Error (RMSE)

Root Mean Squared Error corresponds to the square root of the average of the squared difference between the target value and the value predicted by the regression model. Basically,  $\sqrt{MSE}$ .

$$RMSE = \sqrt{MSE}$$

## Score (Coefficient of Determination)

R<sup>2</sup> Coefficient of determination actually works as a post metric, meaning it's a metric that's calculated using other metrics.

The point of even calculating this coefficient is to answer the question "How much (what %) of the total variation in Y(target) is explained by the variation in X(regression line)

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- Measures how much variance in the target is explained by the model.
- Values range from **0 to 1** (sometimes negative for very poor models).

**Code example:**

```

from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

Actual vs predicted values
y_true = np.array([100, 200, 300, 400])
y_pred = np.array([110, 190, 290, 410])

MSE
mse = mean_squared_error(y_true, y_pred)
print("Mean Squared Error (MSE):", mse)

RMSE
rmse = np.sqrt(mse)
print("Root Mean Squared Error (RMSE):", rmse)

R² Score
r2 = r2_score(y_true, y_pred)
print("R² Score:", r2)

```

Metric	Use Case	Strengths	Limitations
<b>MSE</b>	You want to penalize <b>larger errors more heavily</b>	Sensitive to outliers, good for optimization	Not in original units, hard to interpret
<b>RMSE</b>	You want error in <b>same units</b> as target	Easy to interpret, common in practice	Still penalizes outliers
<b>R² Score</b>	You want to measure <b>how well the model explains variance</b>	Scales from 0 to 1, interpretable	Can be negative, sensitive to outliers

## Cross-Validation

Cross-validation is a resampling procedure used to evaluate machine learning models on a limited data sample. If you have a machine learning model and some data, you want to determine whether your model can generalize well. Typically, you would split your data into a training set and a test set. You train your model with the training set and then evaluate its performance using the test set. However, evaluating the model only once may not be sufficient; there's a possibility that the positive results you obtained are simply due to chance. To gain more confidence in your model's performance, you should evaluate it multiple times.

Cross-validation is primarily used in applied machine learning to estimate how well a model is likely to perform on unseen data. In essence, it allows you to use a limited sample to predict how the model will perform when applied to new data that was not used during its training.

## K-Fold Cross-Validation

K-Fold Cross-Validation is a robust technique used to evaluate the performance of machine learning models. It helps ensure that the model generalizes well to unseen data by using different portions of the dataset for training and testing in multiple iterations.

**Shuffle the dataset randomly.**

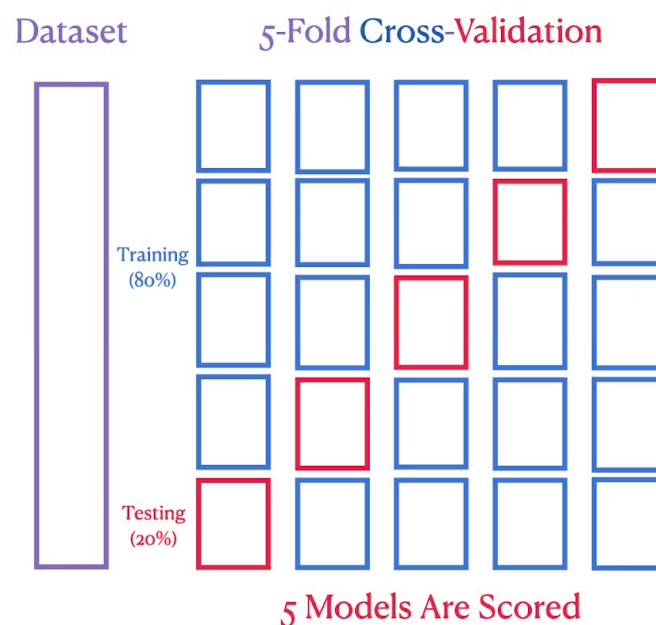
**Split the dataset into k groups**

**For each unique group:**

- Take the group as a holdout or test data set
- Take the remaining groups as a training data set
- Fit a model on the training set and evaluate it on the test set
- Retain the evaluation score and discard the model

**Summarize the skill of the model using the sample of model evaluation scores.**

The image illustrates a 5-Fold Cross-Validation, where the dataset is split into five parts, with each part serving as a test set in one of the five iterations, ensuring each segment is used for both training and testing.



This technique:

- **Reduces variance** in model evaluation.
- Helps detect **overfitting**.
- Gives a more **robust performance estimate**.

**Code example:**

```
from sklearn.model_selection import KFold
k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42)
```

## K-Fold vs. Train-Test Split

Aspect	K-Fold Cross-Validation	Train-Test Split
--------	-------------------------	------------------

Aspect	K-Fold Cross-Validation	Train-Test Split
<b>Data Utilization</b>	Data is divided into multiple folds, ensuring that each data point has a chance to be part of both the training and validation sets across different iterations.	Divide the data into fixed portions for training and testing.
<b>Bias-Variance Tradeoff</b>	It reduces variance as the model is trained multiple times on unseen data; hence, the optimal bias-variance tradeoff is achieved.	There is a chance of high variance with the plain train-test split. This often occurs because the model hugs the training data well and often fails to understand the test data.
<b>Overfitting Risk</b>	There is a low risk of overfitting as the model gets tested across different folds.	Higher risk of overfitting if the train-test split is not representative.
<b>Performance Evaluation</b>	Provides a more reliable and generalized performance estimate.	Performance depends on a single train-test split, which may be biased.

## Conclusion

Evaluating your model is just as important as building it. In this lesson, you've learned how to measure model performance using both **classification** and **regression** metrics. From understanding confusion matrices to calculating accuracy, precision, recall, F1-score, ROC-AUC, and various regression metrics like MSE and  $R^2$ , you now have a solid foundation to **interpret your model's behavior** and **choose the right metric** for the problem you're solving.